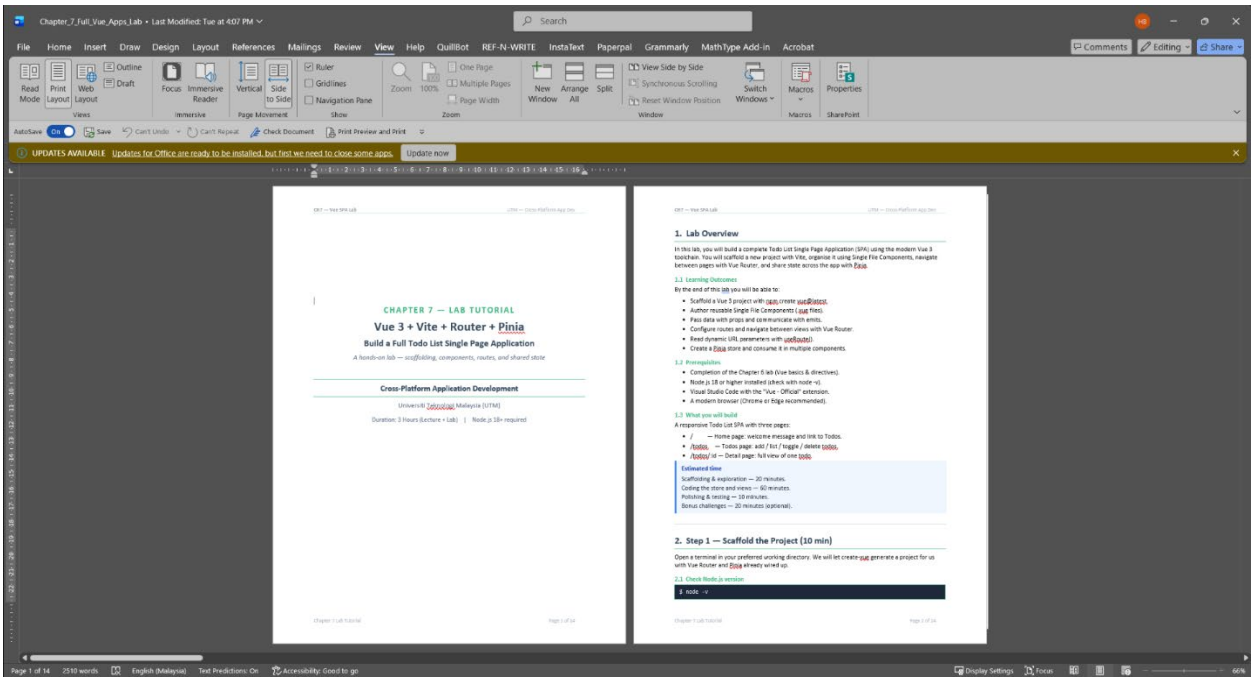


CHAPTER 7 — LAB TUTORIAL

Vue 3 + Vite + Router + Pinia

Build a Full Todo List Single Page Application

A hands-on lab — scaffolding, components, routes, and shared state

Cross-Platform Application Development

Universiti Teknologi Malaysia (UTM)

Duration: | Node.js 18+ required

1. Lab Overview

In this lab, you will build a complete Todo List Single Page Application (SPA) using the modern Vue 3 toolchain. You will scaffold a new project with Vite, organise it using Single File Components, navigate between pages with Vue Router, and share state across the app with Pinia.

1.1 Learning Outcomes

By the end of this lab you will be able to:

- Scaffold a Vue 3 project with npm create vue@latest.
- Author reusable Single File Components (.vue files).
- Pass data with props and communicate with emits.
- Configure routes and navigate between views with Vue Router.
- Read dynamic URL parameters with useRoute().
- Create a Pinia store and consume it in multiple components.

1.2 Prerequisites

- Completion of the Chapter 6 lab (Vue basics & directives).
- Node.js 18 or higher installed (check with node -v).
- Visual Studio Code with the "Vue - Official" extension.
- A modern browser (Chrome or Edge recommended).

1.3 What you will build

A responsive Todo List SPA with three pages:

- / — Home page: welcome message and link to Todos.
- /todos — Todos page: add / list / toggle / delete todos.
- /todos/:id — Detail page: full view of one todo.

> **Estimated time**
>
> Scaffolding & exploration — 20 minutes.
> Coding the store and views — 60 minutes.
> Polishing & testing — 10 minutes.
> Bonus challenges — 20 minutes (optional).

---

2. Step 1 — Scaffold the Project (10 min)

Open a terminal in your preferred working directory. We will let create-vue generate a project for us with Vue Router and Pinia already wired up.

2.1 Check Node.js version

```
$ node -v
```

```
v20.12.0         # any 18+ is fine

$ npm -v
10.5.0
```

If the node command is not found, download the LTS version from https://nodejs.org and restart your terminal.

2.2 Create the project

```
$ npm create vue@latest
```

When prompted, answer the questions like this (press Enter to accept defaults where shown):

```
✓ Project name:                      todo-spa
✓ Add TypeScript?                    › No
✓ Add JSX Support?                   › No
✓ Add Vue Router for SPA?            › Yes      ← important
✓ Add Pinia for state management?    › Yes      ← important
✓ Add Vitest for Unit Testing?       › No
✓ Add End-to-End Testing Solution?   › No
✓ Add ESLint for code quality?       › Yes
✓ Add Prettier for code formatting?  › Yes
```

2.3 Install and run

```
$ cd todo-spa
$ npm install      # installs ~300 packages; takes 1-2 minutes
$ npm run dev
```

You should see something like:

```
  VITE v5.x.x  ready in 432 ms

  ➜  Local:   http://localhost:5173/
  ➜  press h + enter to show help
```

Open http://localhost:5173 in your browser. You should see the default Vue starter page with a Home link and an About link.

> **Keep the dev server running**
>
> Open a second terminal tab for running other commands. Your Vite dev server should stay running throughout the lab — it hot-reloads every time you save a file.

---

3. Step 2 — Explore the Starter Code (10 min)

Open the todo-spa folder in VS Code and take a quick tour. Do not change anything yet.

3.1 Key files

```
todo-spa/
├─ index.html              # app shell — a single <div id="app">
├─ src/
│  ├─ main.js              # creates the app, installs plugins, mounts
│  ├─ App.vue              # root component: <RouterView> lives here
│  ├─ assets/              # global CSS lives here
│  ├─ components/
│  │  └─ HelloWorld.vue    # starter component (you can delete)
│  ├─ router/
│  │  └─ index.js          # route definitions
│  ├─ stores/
│  │  └─ counter.js        # starter Pinia store (reference)
│  └─ views/
│     ├─ HomeView.vue      # renders at /
│     └─ AboutView.vue     # renders at /about
```

Peek inside src/main.js. Notice how createPinia and the router are installed on the Vue app:

```
import './assets/main.css'
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'

const app = createApp(App)

app.use(createPinia())
app.use(router)

app.mount('#app')
```

> **Your mental model**
>
> The whole app renders inside the <RouterView /> that lives in App.vue. As the URL changes, Vue Router swaps the matched view (HomeView, AboutView, etc.) in and out of that spot.

---

4. Step 3 — Create the Todos Pinia Store (15 min)

We will hold all todo data in a single Pinia store so every page can read and modify it.

4.1 Delete the starter store

Remove src/stores/counter.js — we will not need it.

4.2 Create src/stores/todos.js

```
import { defineStore } from 'pinia'
```

```
import { ref, computed } from 'vue'

export const useTodosStore = defineStore('todos', () => {

  // -------- state --------
  const items = ref([
    { id: 1, text: 'Finish Chapter 7 lab', done: false, notes: '' },
    { id: 2, text: 'Commit code to Git',    done: false, notes: '' },
  ])

  // -------- getters --------
  const total      = computed(() => items.value.length)
  const doneCount  = computed(() => items.value.filter(t => t.done).length)
  const activeCount = computed(() => total.value - doneCount.value)

  // -------- actions --------
  function add(text) {
    const clean = text.trim()
    if (!clean) return
    items.value.push({
      id: Date.now(),
      text: clean,
      done: false,
      notes: ''
    })
  }

  function remove(id) {
    items.value = items.value.filter(t => t.id !== id)
  }

  function toggle(id) {
    const t = items.value.find(t => t.id === id)
    if (t) t.done = !t.done
  }

  function updateNotes(id, notes) {
    const t = items.value.find(t => t.id === id)
    if (t) t.notes = notes
  }

  function getById(id) {
    return items.value.find(t => t.id === Number(id))
  }

  return { items, total, doneCount, activeCount,
           add, remove, toggle, updateNotes, getById }
})
```

> **Composition API Pinia — don't forget to return**
>
> With the function syntax of defineStore, only the names you return become part of the public store API. If you call store.add() and it's undefined, check your return statement.

---

5. Step 4 — Build a TodoItem Component (15 min)

We will isolate the markup for a single todo row into its own component. The parent will pass a todo via a prop and listen for a toggle or remove event via emits.

5.1 Create src/components/TodoItem.vue

```
<template>
  <li :class="{ done: todo.done }" class="todo">
    <input type="checkbox"
           :checked="todo.done"
           @change="$emit('toggle', todo.id)">

    <router-link :to="{ name: 'todo-detail', params: { id: todo.id } }"
                 class="text">
      {{ todo.text }}
    </router-link>

    <button class="remove"
            @click="$emit('remove', todo.id)"
            title="Delete">&times;</button>
  </li>
</template>

<script setup>
defineProps({
  todo: { type: Object, required: true }
})

defineEmits(['toggle', 'remove'])
</script>

<style scoped>
.todo {
  display: flex;
  align-items: center;
  gap: .75rem;
  padding: .6rem .75rem;
  border-bottom: 1px solid #E2E8F0;
}
.todo.done .text {
  text-decoration: line-through;
```

```
  color: #94A3B8;
}
.text {
  flex: 1;
  text-decoration: none;
  color: #1F2937;
}
.text:hover { color: #42B883; }
.remove {
  background: none;
  border: none;
  color: #EF4444;
  cursor: pointer;
  font-size: 1.25rem;
}
</style>
```

Scoped styles

<style scoped> rewrites CSS selectors so they only match elements in this component. That is why you can safely use generic class names like .todo or .text across many components without collisions.

6. Step 5 — Build the Todos View (20 min)

6.1 Replace src/views/AboutView.vue with TodosView.vue

Rename (or create) src/views/TodosView.vue with the following:

```
<template>
  <section class="wrap">
    <h1>Todos</h1>

    <form @submit.prevent="submit" class="add-form">
      <input v-model="text"
             placeholder="What needs doing?"
             autofocus>
      <button type="submit"
              :disabled="!text.trim()">Add</button>
    </form>

    <p v-if="store.total === 0" class="empty">
      Nothing yet — add your first todo above.
    </p>

    <ul v-else class="list">
      <TodoItem v-for="t in store.items"
```

```
                :key="t.id"
                :todo="t"
                @toggle="store.toggle"
                @remove="store.remove" />
    </ul>

    <p class="stats">
      {{ store.activeCount }} active ·
      {{ store.doneCount }} done ·
      {{ store.total }} total
    </p>
  </section>
</template>

<script setup>
import { ref } from 'vue'
import { useTodosStore } from '@/stores/todos'
import TodoItem from '@/components/TodoItem.vue'

const store = useTodosStore()
const text  = ref('')

function submit() {
  store.add(text.value)
  text.value = ''
}
</script>

<style scoped>
.wrap { max-width: 520px; margin: 2rem auto; padding: 1rem; }
h1 { color: #35495E; }
.add-form { display: flex; gap: .5rem; margin-bottom: 1rem; }
.add-form input {
  flex: 1; padding: .6rem; border: 1px solid #CBD5E1;
  border-radius: 6px; font-size: 1rem;
}
.add-form button {
  background: #42B883; color: #FFFFFF;
  border: none; padding: .6rem 1rem; border-radius: 6px;
  font-weight: 600; cursor: pointer;
}
.add-form button:disabled { background: #CBD5E1; cursor: not-allowed; }

.list { list-style: none; padding: 0;
        background: #FFFFFF; border-radius: 12px;
        box-shadow: 0 4px 12px rgba(0,0,0,.05); }
.empty  { color: #64748B; font-style: italic; }
.stats  { margin-top: 1rem; color: #64748B; font-size: .9rem;
          text-align: center; }
```

```
</style>
```

> **What is @/ ?**
>
> The @ alias is pre-configured by Vite to point at the src/ folder. Writing @/stores/todos is equivalent to ../stores/todos, but it always works regardless of how deep the current file is.

---

7. Step 6 — Build the Detail View (15 min)

The detail view reads the :id from the URL, looks up the matching todo in the store, and lets you edit the notes field.

7.1 Create src/views/TodoDetailView.vue

```
<template>
  <section class="wrap">
    <router-link to="/todos" class="back">&larr; Back</router-link>

    <div v-if="todo" class="card" :class="{ done: todo.done }">
      <h1>{{ todo.text }}</h1>
      <p class="status">
        Status:
        <strong>{{ todo.done ? 'Completed' : 'Active' }}</strong>
      </p>

      <label>
        Notes
        <textarea v-model="notes" rows="4"
                  placeholder="Add some context..."></textarea>
      </label>

      <div class="actions">
        <button class="save" @click="saveNotes">Save notes</button>
        <button class="toggle" @click="store.toggle(todo.id)">
          Mark as {{ todo.done ? 'active' : 'done' }}
        </button>
      </div>
    </div>

    <div v-else class="missing">
      <p>Todo not found.</p>
      <router-link to="/todos">Back to list</router-link>
    </div>
  </section>
</template>

<script setup>
```

```
import { ref, watch, computed } from 'vue'
import { useRoute } from 'vue-router'
import { useTodosStore } from '@/stores/todos'

const route = useRoute()
const store = useTodosStore()

const todo = computed(() => store.getById(route.params.id))
const notes = ref(todo.value?.notes ?? '')

// keep textarea in sync if user navigates to another todo
watch(todo, t => { notes.value = t?.notes ?? '' })

function saveNotes() {
  if (todo.value) store.updateNotes(todo.value.id, notes.value)
}
</script>

<style scoped>
.wrap { max-width: 520px; margin: 2rem auto; padding: 1rem; }
.back { color: #42B883; text-decoration: none; font-weight: 600; }
.card {
  background: #FFFFFF; margin-top: 1rem;
  border-radius: 12px; padding: 1.25rem;
  box-shadow: 0 4px 12px rgba(0,0,0,.05);
}
.card.done h1 { text-decoration: line-through; color: #94A3B8; }
.status  { color: #64748B; margin-bottom: 1rem; }
label    { display: block; margin-top: 1rem; color: #35495E; }
textarea {
  width: 100%; margin-top: .25rem; padding: .5rem;
  border: 1px solid #CBD5E1; border-radius: 6px;
  font-family: inherit; font-size: 1rem;
}
.actions { display: flex; gap: .5rem; margin-top: 1rem; }
.save, .toggle {
  padding: .5rem .9rem; border: none;
  border-radius: 6px; cursor: pointer; font-weight: 600;
}
.save   { background: #42B883; color: #FFFFFF; }
.toggle { background: #E2E8F0; color: #1F2937; }
.missing { margin-top: 2rem; color: #EF4444; }
</style>
```

8. Step 7 — Wire Up the Routes (5 min)

8.1 Update src/router/index.js

Replace the contents of this file with:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView  from '@/views/HomeView.vue'
import TodosView from '@/views/TodosView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/',       name: 'home',         component: HomeView  },
    { path: '/todos',  name: 'todos',        component: TodosView },
    {
      path: '/todos/:id',
      name: 'todo-detail',
      // lazy-loaded — only fetched when the route is visited
      component: () => import('@/views/TodoDetailView.vue')
    },
    {
      path: '/:pathMatch(.*)*',
      name: 'not-found',
      component: () => import('@/views/HomeView.vue')
    }
  ]
})

export default router
```

8.2 Update src/views/HomeView.vue

Keep the home view simple — a welcome message and a big button to the todos page:

```
<template>
  <section class="hero">
    <h1>Todo SPA</h1>
    <p>Built with Vue 3, Vite, Vue Router and Pinia.</p>
    <router-link to="/todos" class="cta">Open my todos &rarr;</router-link>
  </section>
</template>

<style scoped>
.hero { text-align: center; padding: 4rem 1rem; }
h1    { font-size: 3rem; color: #35495E; margin: 0; }
p     { color: #64748B; }
.cta  {
  display: inline-block; margin-top: 1.5rem;
  background: #42B883; color: #FFFFFF;
  padding: .8rem 1.4rem; border-radius: 8px;
  text-decoration: none; font-weight: 600;
```

```
}
.cta:hover { background: #33A06F; }
</style>
```

8.3 Simplify App.vue

Remove the default header content in App.vue and keep only the <RouterView />, plus a small top navigation bar:

```
<template>
  <header>
    <nav>
      <router-link to="/">Home</router-link>
      <router-link to="/todos">Todos</router-link>
    </nav>
  </header>

  <main>
    <RouterView />
  </main>
</template>

<script setup>
import { RouterLink, RouterView } from 'vue-router'
</script>

<style>
body { margin: 0; font-family: system-ui, sans-serif;
       background: #F1F5F9; color: #1F2937; }
header { background: #35495E; padding: 1rem; }
nav    { max-width: 520px; margin: 0 auto; display: flex; gap: 1rem; }
nav a  { color: #FFFFFF; text-decoration: none; font-weight: 600; }
nav a.router-link-active { color: #42B883; }
</style>
```

9. Step 8 — Test & Verify (5 min)

Save all files. Vite should have hot-reloaded automatically. In the browser:

1. Open http://localhost:5173/ — you should see the landing page.
2. Click "Open my todos" — you land on /todos with two starter todos.
3. Add three new todos using the input and button (or Enter).
4. Toggle the checkboxes — the text should strike through when done.
5. Click a todo's text — you navigate to /todos/<id> (the detail page).
6. Type something in the Notes field and click Save.

7. Click Back, then return to the same todo — the notes should persist (while the dev server is running).
8. Delete a todo with the × button — it disappears from the list.
9. Try an unknown URL like /todos/999 — the detail view should show "Todo not found."

> **State resets on full page reload**
>
> Pinia state lives in memory. If you reload the browser tab, the starter list returns. Persisting across reloads is a bonus challenge below.

---

10. Bonus Challenges (Optional)

Challenge A — Filters (easy)

- Add three tabs on /todos: All, Active, Done.
- Use a computed property in the TodosView that filters store.items based on the selected tab.

Challenge B — Persist to localStorage (medium)

- In src/stores/todos.js, read the initial items from localStorage if present.
- Use watch(items, newItems => localStorage.setItem('todos', JSON.stringify(newItems)), { deep: true }).
- Reload the browser — todos should persist.

Challenge C — 404 page (medium)

- Create src/views/NotFoundView.vue with a friendly message.
- Point the catch-all route to it instead of HomeView.

Challenge D — Navigation guard (harder)

- Add a beforeEach guard in router/index.js that redirects to /todos if the user goes to /todos/undefined or /todos/NaN.

Challenge E — Deploy

- Run npm run build — examine the dist/ folder.
- Deploy to Netlify Drop (https://app.netlify.com/drop) — drag and drop the dist folder.
- Share your public URL with the instructor.

— End of Chapter 7 Lab —

Good luck and have fun building!